

Exercício – Padrão Observer (iFood)

Objetivo

Implementar o padrão de projeto **Observer** em Java, simulando o funcionamento de um sistema de pedidos semelhante ao iFood, onde diferentes partes do sistema são notificadas automaticamente quando o status de um pedido é atualizado.

Em aplicativos de delivery (como iFood, Uber Eats, etc.), um pedido passa por vários estados ao longo do tempo. Cada mudança de estado precisa ser comunicada a diferentes interessados:

- Cliente (acompanha o pedido)
- Restaurante (prepara o pedido)
- Entregador (realiza a entrega)

Esses componentes não devem estar diretamente acoplados entre si. Em vez disso, o sistema deve utilizar o padrão **Observer**, permitindo que os interessados sejam notificados automaticamente sempre que o estado do pedido mudar.

O QUE DEVERÁ SER IMPLEMENTADO:

1. Estados do pedido

Crie um enum chamado StatusPedido com os seguintes valores:

- RECEBIDO
- PREPARANDO
- SAIU_PARA_ENTREGA
- ENTREGUE

2. Interface Subject

Crie uma interface Subject com os métodos:

```
void registerObserver(Observer observer);  
void removeObserver(Observer observer);  
void notifyObservers();
```

3. Interface Observer

Crie uma interface Observer com o método:

```
void update(Pedido pedido);
```

4. Classe Pedido (Subject concreto)

Implemente uma classe Pedido que:

- Implementa a interface Subject
- Possui:

- Uma lista de observadores (List<Observer>)
- Um atributo StatusPedido status

Comportamentos:

- Permitir registrar e remover observadores
- Notificar todos os observadores quando o status mudar
- Método:


```
void setStatus(StatusPedido status)
```

Esse método deve:

1. Atualizar o status
2. Chamar notifyObservers()

5. Observadores concretos

Implemente as seguintes classes que devem implementar Observer:

a) ClienteApp

- Simula o aplicativo do cliente
- Deve imprimir:

Cliente recebeu notificação: Pedido está <STATUS>

b) RestaurantePainel

- Simula o painel do restaurante
- Deve imprimir:

Restaurante recebeu atualização: Pedido está <STATUS>

c) EntregadorApp

- Simula o aplicativo do entregador
- Deve imprimir:

Entregador recebeu atualização: Pedido está <STATUS>

6. Classe principal (Main)

Crie uma classe Main que:

1. Instancia um objeto Pedido
2. Instancia os observadores:
 - ClienteApp
 - RestaurantePainel
 - EntregadorApp
3. Registra todos os observadores no pedido
4. Simula mudanças de estado:


```
pedido.setStatus(StatusPedido.PREPARANDO);
pedido.setStatus(StatusPedido.SAIU_PARA_ENTREGA);
pedido.setStatus(StatusPedido.ENTREGUE);
```

Saída esperada (exemplo)

Cliente recebeu notificação: Pedido está PREPARANDO
 Restaurante recebeu atualização: Pedido está PREPARANDO
 Entregador recebeu atualização: Pedido está PREPARANDO

Cliente recebeu notificação: Pedido está SAIU_PARA_ENTREGA
Restaurante recebeu atualização: Pedido está SAIU_PARA_ENTREGA
Entregador recebeu atualização: Pedido está SAIU_PARA_ENTREGA

Cliente recebeu notificação: Pedido está ENTREGUE
Restaurante recebeu atualização: Pedido está ENTREGUE
Entregador recebeu atualização: Pedido está ENTREGUE